

Sales Forecasting Using ARIMA

Import Required Libraries

```
In [1]: import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from statsmodels.tsa.arima.model import ARIMA

from sklearn.metrics import mean_absolute_error, mean_squared_error

import warnings
warnings.filterwarnings("ignore")
```

Read the Dataset

```
In [2]: df = pd.read_excel("Sales_Forecasting_Codec.xlsx")

df.head()
```

```
Out[2]:
```

	Date	StoreID	Region	Category	Product	SalesRep	UnitsSold	UnitPrice	Discount
0	2021-01-01	1	South	Office Supplies	Laptop	REP025	217	410.76	
1	2021-01-01	2	South	Technology	Mouse	REP001	130	666.81	
2	2021-01-01	3	South	Office Supplies	Mouse	REP007	121	255.64	
3	2021-01-01	4	North	Technology	Phone	REP001	267	243.20	
4	2021-01-01	5	East	Office Supplies	Keyboard	REP003	151	274.46	

Dataset Information

```
In [3]: df.shape
```

```
Out[3]: (14610, 18)
```

```
In [4]: df.columns
```

```
Out[4]: Index(['Date', 'StoreID', 'Region', 'Category', 'Product', 'SalesRep',
              'UnitsSold', 'UnitPrice', 'DiscountPercent', 'Revenue', 'Cost',
              'Profit', 'Promotion', 'Holiday', 'CustomerFootfall', 'Inventory',
              'StockOut', 'CustomerRating'],
              dtype='object')
```

```
In [5]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14610 entries, 0 to 14609
Data columns (total 18 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Date                  14610 non-null  object
1   StoreID               14610 non-null  int64
2   Region                14610 non-null  object
3   Category              14610 non-null  object
4   Product               14610 non-null  object
5   SalesRep              14610 non-null  object
6   UnitsSold             14610 non-null  int64
7   UnitPrice             14610 non-null  float64
8   DiscountPercent       14610 non-null  int64
9   Revenue               14610 non-null  float64
10  Cost                  14610 non-null  float64
11  Profit                14610 non-null  float64
12  Promotion              14610 non-null  object
13  Holiday                14610 non-null  object
14  CustomerFootfall      14610 non-null  int64
15  Inventory              14610 non-null  int64
16  StockOut              14610 non-null  object
17  CustomerRating        14610 non-null  int64
dtypes: float64(4), int64(6), object(8)
memory usage: 2.0+ MB
```

```
In [6]: df.describe()
```

```
Out[6]:
```

	StoreID	UnitsSold	UnitPrice	DiscountPercent	Revenue	
count	14610.00000	14610.000000	14610.000000	14610.000000	14610.000000	14610
mean	5.50000	138.987201	405.725874	12.471253	49426.836034	3455
std	2.87238	71.568271	226.338316	8.523724	40679.665917	2888
min	1.00000	10.000000	15.070000	0.000000	234.920000	15
25%	3.00000	78.000000	210.360000	5.000000	16467.837500	1126
50%	5.50000	139.000000	404.455000	10.000000	38676.565000	2680
75%	8.00000	199.000000	598.867500	20.000000	73034.247500	5107
max	10.00000	310.000000	800.000000	25.000000	224895.000000	16404

```
In [7]: df.isnull().sum()
```

Out[7]:

	0
Date	0
StoreID	0
Region	0
Category	0
Product	0
SalesRep	0
UnitsSold	0
UnitPrice	0
DiscountPercent	0
Revenue	0
Cost	0
Profit	0
Promotion	0
Holiday	0
CustomerFootfall	0
Inventory	0
StockOut	0
CustomerRating	0

dtype: int64

Data Preprocessing

```
In [8]: df["Date"] = pd.to_datetime(df["Date"])
```

```
In [9]: df = df.sort_values("Date")
```

```
In [10]: sales = df.groupby("Date")["Revenue"].sum()

sales.head()
```

Out[10]:

Revenue	
Date	
2021-01-01	581958.40
2021-01-02	381088.82
2021-01-03	543573.73
2021-01-04	400590.56
2021-01-05	421499.63

dtype: float64

Exploratory Data Analysis (EDA)

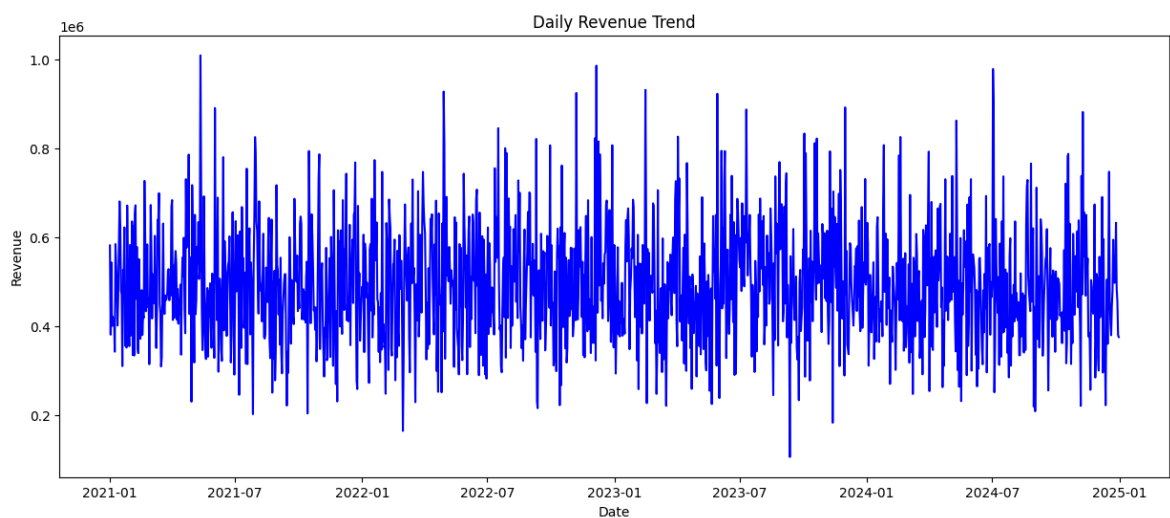
Trend Analysis

```
In [11]: plt.figure(figsize=(15,6))

plt.plot(sales, color='blue')

plt.title("Daily Revenue Trend")
plt.xlabel("Date")
plt.ylabel("Revenue")

plt.show()
```



Seasonal Analysis

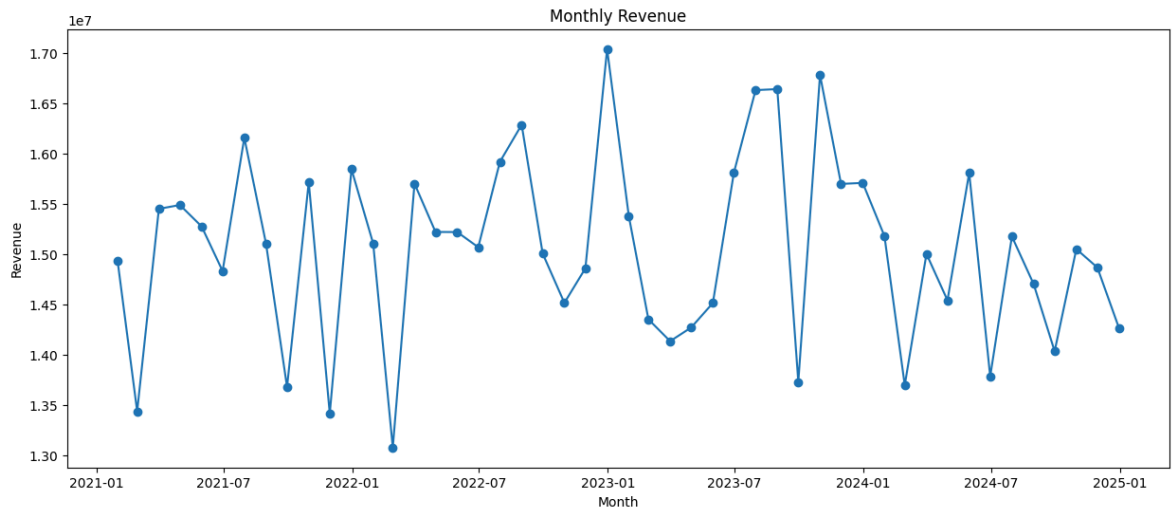
```
In [12]: monthly_sales = sales.resample("ME").sum()

plt.figure(figsize=(15,6))

plt.plot(monthly_sales, marker='o')

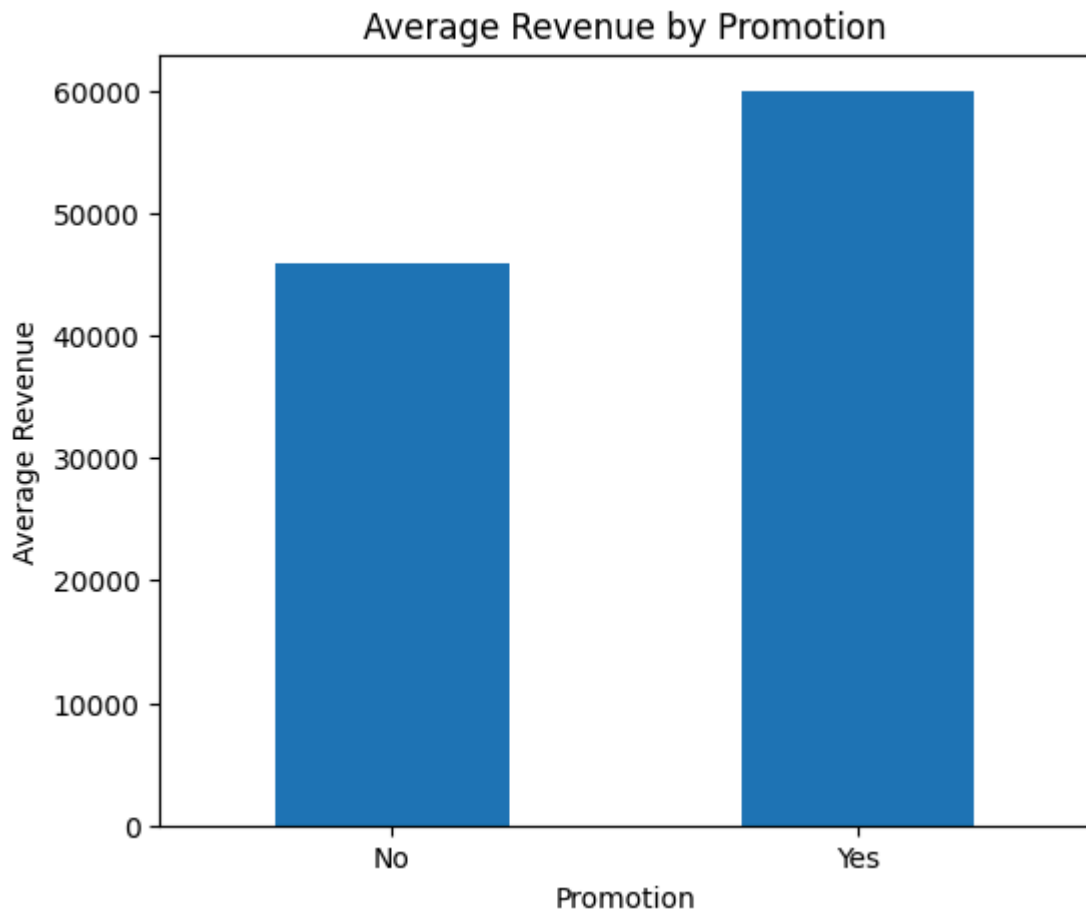
plt.title("Monthly Revenue")
plt.xlabel("Month")
```

```
plt.ylabel("Revenue")  
  
plt.show()
```



Promotion Analysis

```
In [13]: promo_sales = df.groupby("Promotion")["Revenue"].mean()  
  
plt.figure(figsize=(6,5))  
  
promo_sales.plot(kind="bar")  
  
plt.title("Average Revenue by Promotion")  
plt.xlabel("Promotion")  
plt.ylabel("Average Revenue")  
  
plt.xticks(rotation=0)  
  
plt.show()
```



Holiday Analysis

```
In [14]: holiday_sales = df.groupby("Holiday")["Revenue"].mean()

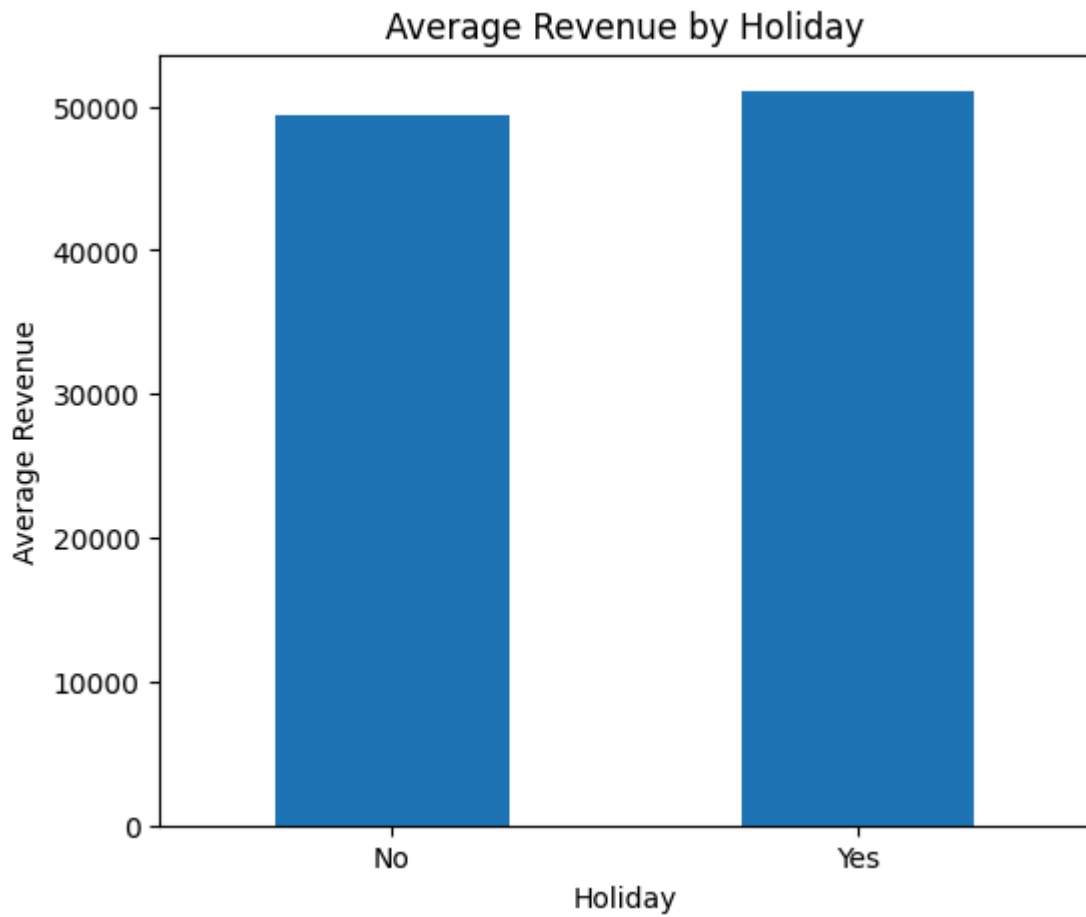
plt.figure(figsize=(6,5))

holiday_sales.plot(kind="bar")

plt.title("Average Revenue by Holiday")
plt.xlabel("Holiday")
plt.ylabel("Average Revenue")

plt.xticks(rotation=0)

plt.show()
```



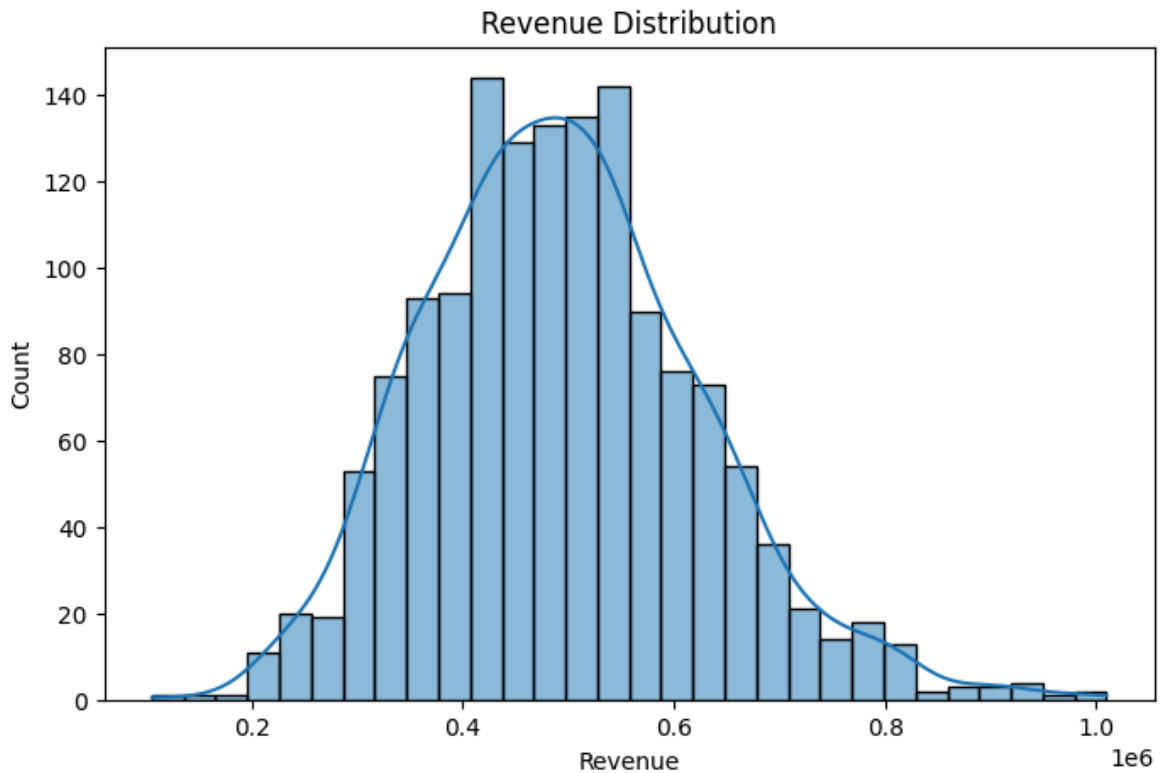
Revenue Distribution

```
In [15]: plt.figure(figsize=(8,5))

sns.histplot(sales, bins=30, kde=True)

plt.title("Revenue Distribution")
plt.xlabel("Revenue")

plt.show()
```



ARIMA Model

Step 1: Train-Test Split

```
In [16]: train = sales[:-30]
test = sales[-30:]

print("Training Data:", train.shape)
print("Testing Data:", test.shape)
```

Training Data: (1431,)

Testing Data: (30,)

Step 2: Build the ARIMA Model

```
In [17]: model = ARIMA(train, order=(5,1,0))

model_fit = model.fit()
```

Step 3: Forecast Future Sales

```
In [18]: forecast = model_fit.forecast(steps=30)

forecast.head()
```

Out[18]:

	predicted_mean
2024-12-02	441167.384576
2024-12-03	402769.270166
2024-12-04	428464.164294
2024-12-05	437410.571134
2024-12-06	422522.520530

dtype: float64

Model Evaluation

```
In [19]: mae = mean_absolute_error(test, forecast)

rmse = np.sqrt(mean_squared_error(test, forecast))

print("Mean Absolute Error (MAE):", round(mae,2))
print("Root Mean Squared Error (RMSE):", round(rmse,2))
```

Mean Absolute Error (MAE): 100076.93

Root Mean Squared Error (RMSE): 126881.76

Forecast vs Actual Sales

```
In [20]: plt.figure(figsize=(14,6))

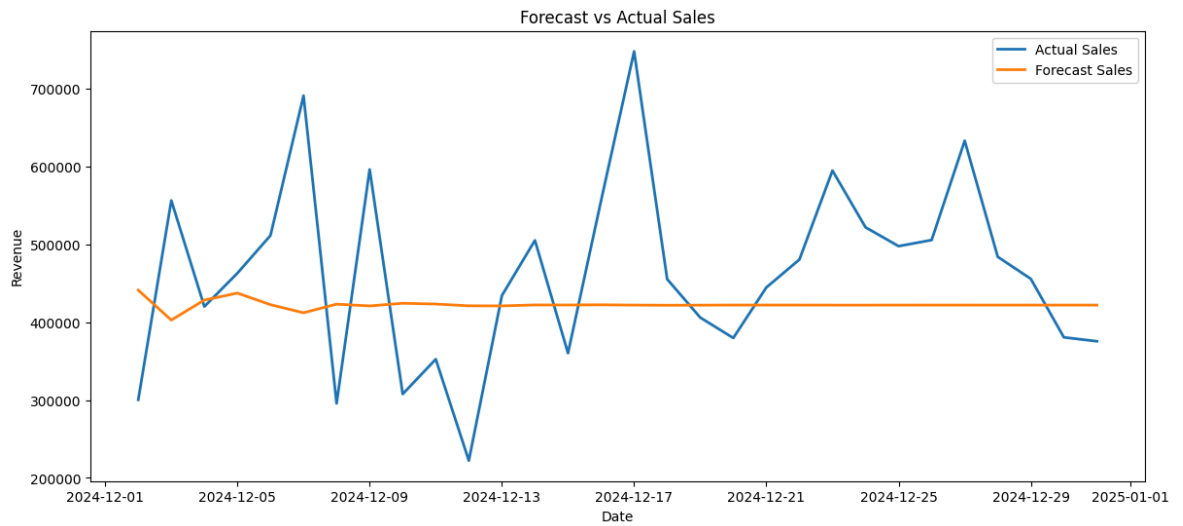
plt.plot(test.index, test, label="Actual Sales", linewidth=2)

plt.plot(test.index, forecast, label="Forecast Sales", linewidth=2)

plt.title("Forecast vs Actual Sales")
plt.xlabel("Date")
plt.ylabel("Revenue")

plt.legend()

plt.show()
```



Conclusion

- This project used historical sales data to forecast future revenue using the ARIMA time-series forecasting model.
- The dataset was analyzed to identify trend, seasonal, and promotional patterns before building the forecasting model. The model's performance was evaluated using Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE).
- The forecast was compared with the actual sales values to visualize the model's predictive performance.